Terminator

A Minimal X11 Terminal Emulator and Shell in C

Sreeram

November 3, 2026

Contents

1 What this project is **2**

2 Dependencies **2**

3 Project layout **2**

4 Installation of dependencies **2**

5 Building and running **3**

6 Using the shell **5**

7 Features implemented **5**

8 Design notes **6**

9 Troubleshooting **7**

1 What this project is

Terminator is a self-contained terminal emulator and shell written in C for Linux. It draws its own X11 window, runs commands typed by the user as child processes, captures their output, and displays the result *inside the X11 window*. No console is used for input or output once the program starts.

The shell supports:

- Multiple tabs with independent working directories.
- Pipelines of arbitrary length (`a | b | c`).
- Input and output redirection (`<`, `>`).
- Built-ins: `cd`, `pwd`, `exit`, `history`, `jobs`, `fg`, `bg`, `multiWatch`.
- Quoting, `$VAR`, `$?`, `$(...)`, backticks, and line continuation.
- Multi-line commands with a `>` continuation prompt.
- Ctrl+C and Ctrl+Z for foreground jobs.
- Background jobs (`cmd &`).
- Searchable command history (Ctrl+R) and arrow-key browsing.
- Tab completion of file names.
- Running simple shell scripts via `source` and `bash`.

2 Dependencies

Only the X11 client library and a C compiler are required.

Package	Purpose
`gcc`	C compiler
`make`	Launcher invocation
`libx11-dev` (Debian/Ubuntu)	X11 client library headers
`libx11-devel` (Fedora/RHEL)	X11 client library headers

3 Project layout

```
terminator.c    The complete shell implementation (single file)
make.c          Compiler-and-runner driver; compiled to ./make
install.sh      Installs build dependencies
make            Binary produced from make.c
README.tex      This document
~/.terminatorrc Optional key-binding overrides (not required)
```

4 Installation of dependencies

Automatic (recommended): `install.sh`

Save the following as `install.sh`, make it executable with `chmod +x install.sh`, and run it once as `./install.sh` (or with `sudo ./install.sh` if your distro requires root for package installation).

```
#!/bin/sh
# install.sh - install build dependencies for Terminator

set -e
```

```
if command -v apt-get >/dev/null 2>&1; then
    echo "Detected Debian/Ubuntu (apt)"
    apt-get update
    apt-get install -y build-essential libx11-dev

elif command -v dnf >/dev/null 2>&1; then
    echo "Detected Fedora/RHEL (dnf)"
    dnf install -y gcc make libx11-devel

elif command -v yum >/dev/null 2>&1; then
    echo "Detected CentOS/RHEL (yum)"
    yum install -y gcc make libx11-devel

elif command -v pacman >/dev/null 2>&1; then
    echo "Detected Arch Linux (pacman)"
    pacman -Sy --noconfirm gcc make libx11

elif command -v zypper >/dev/null 2>&1; then
    echo "Detected openSUSE (zypper)"
    zypper --non-interactive install gcc make libX11-devel

elif command -v apk >/dev/null 2>&1; then
    echo "Detected Alpine Linux (apk)"
    apk add --no-cache gcc make musl-dev libx11-dev

else
    echo "Could not detect package manager."
    echo "Please install gcc, make and the X11 development headers"
    echo "for your distribution, then re-run this script."
    exit 1
fi

echo "Dependencies installed."
```

Manual installation

Debian / Ubuntu:

```
sudo apt update
sudo apt install build-essential libx11-dev
```

Fedora / RHEL:

```
sudo dnf install gcc make libx11-devel
```

Arch Linux:

```
sudo pacman -S gcc make libx11
```

5 Building and running

The intended workflow is a one-command compile-and-run launcher. `make.c` compiles `terminator.c` and executes the resulting binary, forwarding any `useProcess` flag.

The launcher: `make.c` (old-school style)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

/* check_deps - return 1 if gcc and libX11 are usable, 0 otherwise */
check_deps()
{
    if (system("which gcc >/dev/null 2>&1") != 0) {
        fprintf(stderr, "gcc is not installed.\n");
        fprintf(stderr, "Run ./install.sh (or install gcc manually).\n");
        return 0;
    }
    if (system("echo 'int main(){return 0;}' | "
               "gcc -x c - -lX11 -o /tmp/.tsrchk >/dev/null 2>&1") != 0) {
        fprintf(stderr, "libX11 development files are missing.\n");
        fprintf(stderr, "Run ./install.sh (or install libx11-dev).\n");
        return 0;
    }
    system("rm -f /tmp/.tsrchk");
    return 1;
}

/* build_term - compile terminator.c into a.out */
build_term()
{
    if (system("gcc terminator.c -lX11 -w -o a.out") != 0) {
        fprintf(stderr, "Compilation of terminator.c failed.\n");
        return 0;
    }
    return 1;
}

/* run_term - execute the compiled binary, forwarding useProcess flag */
run_term(argc, argv)
int  argc;
char *argv[];
{
    char cmd[256];
    if (argc > 1 &&
        (strcmp(argv[1], "useProcess=True") == 0 ||
         strcmp(argv[1], "useProcess=1") == 0))
        sprintf(cmd, "./a.out %s", argv[1]);
    else
        sprintf(cmd, "./a.out");
    return system(cmd);
}

main(argc, argv)
int  argc;
char *argv[];
{
    if (!check_deps()) return 1;
    if (!build_term()) return 1;
    return run_term(argc, argv) == 0 ? 0 : 1;
}
```

Build the launcher once

```
gcc make.c -o make
```

Run in thread mode (default)

```
./make
```

Run in process mode

```
./make useProcess=True
```

Both modes accept the same commands and behave identically from the user's point of view. The `useProcess` flag is parsed for compatibility with the specification.

6 Using the shell

After launching, an X11 window opens with a `$` prompt. Everything — input, output, prompts — appears inside the window.

Example session

```
$ ls
abc.txt  def.txt  myprog.c
$ echo hello | wc -c
6
$ echo "Files in $(basename $(pwd)): $(ls | wc -l)"
Files in home: 3
$ sleep 100 &
[1] 12345
$ jobs
[1] Running sleep 100
$ history
    1  ls
    2  echo hello | wc -c
    3  jobs
$ sort < input.txt > sorted.txt
$ exit
```

Keyboard shortcuts

7 Features implemented

- X11 window with per-tab text buffer and immediate redraw.
- Multi-line input with a `>` continuation prompt when a line ends with \ or an unclosed quote.
- Expansion of `$VAR`, `${VAR}`, `$?`, `$(...)`, and backticks.
- Redirection of stdin and stdout; missing input files are reported and the command is not run.
- Pipelines of any length with the final stage captured into the tab.
- Background jobs (`&`); `jobs`, `fg`, `bg` built-ins.
- `multiWatch ["cmd1", "cmd2", ...]` re-runs each command every two seconds and prefixes output with the command name and a Unix timestamp; Ctrl+C stops all children.

Key	Action
Enter	Run the command
Backspace	Delete previous character
Left/Right	Move cursor within the line
Up/Down	Browse history
Tab	Complete file name
Ctrl+A / Ctrl+E	Jump to start / end of line
Ctrl+U / Ctrl+K	Delete to start / end of line
Ctrl+L	Clear the screen
Ctrl+R	Search history
Ctrl+C	Interrupt foreground command
Ctrl+Z	Suspend foreground command
Ctrl+D	Exit the shell
Ctrl+T	Open a new tab
Ctrl+W	Close current tab
Ctrl+1 ... Ctrl+8	Switch to tab
PageUp/PageDown	Scroll the output

- Searchable history via Ctrl+R; `history` prints the most recent 1,000 entries.
- File-name completion with single-match, longest-prefix and numbered-list behaviour.
- Simple script interpreter used by `source` and `bash`: assignments, `if`/`elif`/ `else`/`fi` with `-eq`, `-gt`, `=`, `-f`, `-d`, and `exit [n]`.

8 Design notes

Text buffer and redraw

Each tab owns a fixed-size character buffer. `buf_add()` appends strings, dropping the oldest line when the buffer fills. `redraw()` splits the buffer on newlines, keeps the last `VISIBLE` lines (adjusted by the scroll offset), and draws them with `XDrawString`. The current input line is never stored in the buffer — it is drawn fresh on every redraw, which prevents the prompt from duplicating.

Command execution and output capture

External commands use `fork()` + `execvp()`. Before forking, the shell creates a pipe and, in the child, redirects stdout and stderr to its write end. The parent reads from the read end and appends the bytes to the tab's screen buffer. Redirected output (`> file`) bypasses the pipe.

Pipelines split the command line on `|` and fork one child per stage; the final stage is connected to a dedicated capture pipe whose read end is drained into the tab.

Signals and job control

The GUI event loop detects Ctrl+C and Ctrl+Z as ordinary key events. When they arrive, the shell calls `kill()` on the foreground child. Stopped children are recorded in a jobs table, reaped by `waitpid(WNOHANG | WUNTRACED)` from the main loop. `fg` and `bg` send `SIGCONT`.

History

History is a fixed-size array inside the process. New entries evict the oldest. Ctrl+R performs a linear back-to-front scan and loads the most recent match into the input buffer.

Completion

Completion reads the current working directory with `opendir`/`readdir`. Depending on the match count it completes uniquely, extends to the longest common prefix, or lists candidates.

9 Troubleshooting

- `Cannot open display` — `DISPLAY` is not set. Run from inside an X session.
- `BadMatch` **on** `X_SetInputFocus` — prevented by only focusing the window after `MapNotify`.
- **No input reaches the window** — click inside the window once; input focus is reset on every mouse press.
- **Output appears in the launching terminal** — verify you are running the binary produced by `./make`.
- `libX11 not found` **at build time** — run `./install.sh` first.